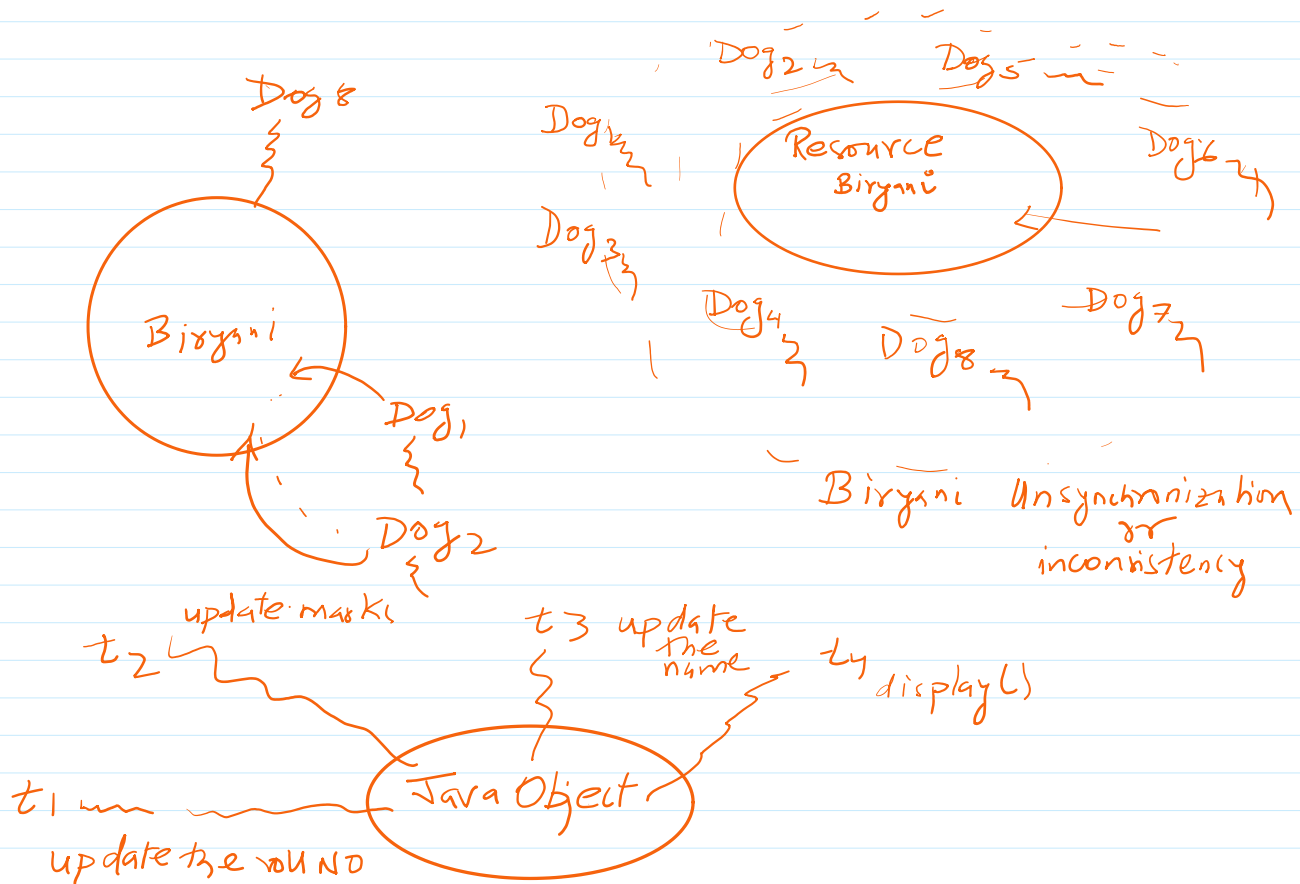


Synchronization

16 March 2026 18:26



Data inconsistency Problem

'synchronized' is a modifier applicable only for method and block a part for classes and variables.

If multiple threads are trying to operate simultaneously on the same Java object there may be chance of data inconsistency problem.

to Java Object, to overcome this problem we should go for "synchronized" keyword. If your method block are declared as "synchronized" then at a time only one thread is allowed to execute. That method is locked on the given object, so that data inconsistency problem will be resolved.

The main advantage of "synchronized" keyword is we can resolve data inconsistency problem. But the main disadvantage of "synchronized" keyword is it increases waiting time for other threads and creates

inconsistency problem. But the main disadvantage of synchronized keyword is it increases waiting time for other threads and creates performance problem. Hence, if there is no specific requirement then it is not recommended to use "synchronized" keyword.

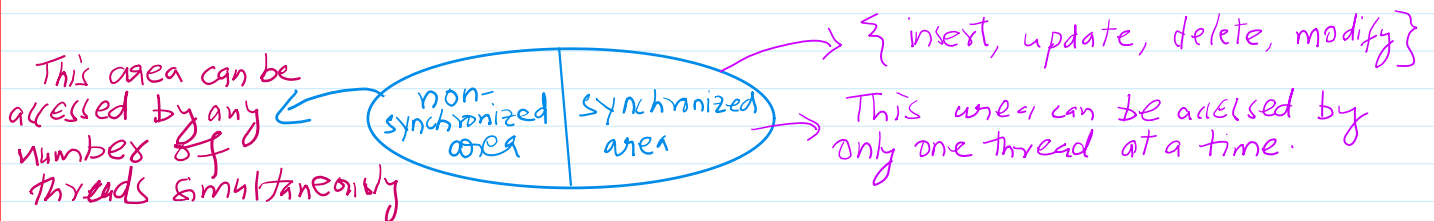
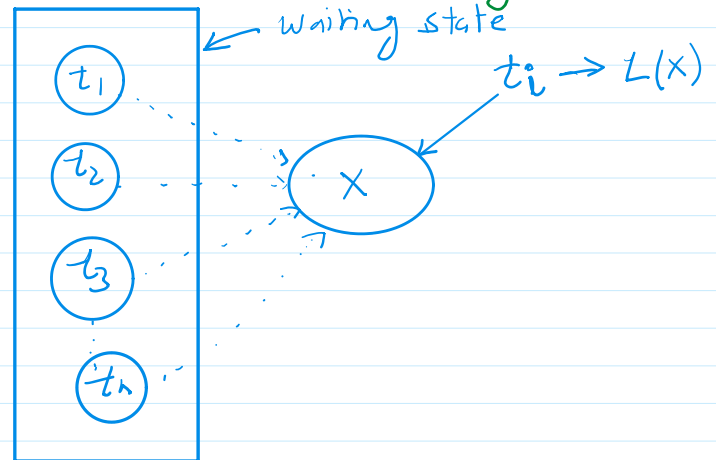
Internally synchronization concept is implemented using lock.

Every object in Java has a unique lock, whenever we are using 'synchronized' keyword, then only lock concept come into the picture. If thread want to

execute synchronized method on the given object, first it has to get the lock of that object, once thread got the lock, then it is allowed to execute any synchronized method on that object.

Once method execution completes automatically thread releases the lock. Acquiring and releasing lock internally taken care by JVM and programmer is not responsible for this activity.

```
class X {
    sync ... m1() { }
    sync ... m2() { }
    sync ... m3() { }
}
```



While a thread executing synchronized method on the given object, the remaining thread are not allowed to execute any synchronized method

the remaining threads are not allowed to execute any synchronized method simultaneously on the same object. But remaining threads are allowed to execute non-synchronized method simultaneously.

class X {

synchronized Area {

whenever we are performing update operation
(add/remove/delete/replace(modify))

i.e., where the state of the object is changing

}

non-synchronized Area {

where the object state won't be changing
like read operation.

}

}

Example :

class ReservationSystem {

(Non-synchronized) checkAvailability() {

just read operation

}

(synchronized) bookTicket() {

update database,

}

}

```
package ThreadSynchronization;
public class Display {
    //non-synchronized
    public void wish(String name){
        for (int i = 0; i < 10; i++) {
            System.out.println("Good morning");
            try {
```

```

        Thread.sleep(1000);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    System.out.println(name);
}
}
}

```

```

package ThreadSynchronization;
public class ThreadSync extends Thread{
    private Display d;
    private String name;
    public ThreadSync(Display d, String name){
        this.d = d;
        this.name = name;
    }
    @Override
    public void run() {
        d.wish(name);
    }
}

```

```

package ThreadSynchronization;
public class SynchronizedDemo {
    public static void main(String[] args) {
        Display d = new Display();
        ThreadSync ts1 = new ThreadSync(d, "Rohit");
        ThreadSync ts2 = new ThreadSync(d, "Virat");
        ts1.start();
        ts2.start();
    }
}

```

Output:

```

Good morning
Good morning
Rohit
Good morning
Virat
Good morning
Virat
Good morning
Rohit
Good morning
Virat
Good morning
Rohit
Good morning
Virat
Good morning
Rohit
Good morning
Rohit
Good morning

```


Virat
Good morning
Virat
Good morning
Rohit
Good morning
Virat
Good morning
Rohit
Good morning
Virat
Good morning
Rohit
Good morning
Virat
Good morning
Rohit
Good morning
Virat
Good morning
Rohit

```
package ThreadSynchronization;
public class Display {
    //synchronized
    public synchronized void wish(String name){
        for (int i = 0; i < 10; i++) {
            System.out.println("Good morning");
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            System.out.println(name);
        }
    }
}
```

Output is predictable when threads are working on same object.

```
package ThreadSynchronization;
public class SynchronizedDemo {
    public static void main(String[] args) {
        Display d = new Display();
        ThreadSync ts1 = new ThreadSync(d, "Rohit");
        ThreadSync ts2 = new ThreadSync(d, "Virat");
        ThreadSync ts3 = new ThreadSync(d, "Surya");
        ThreadSync ts4 = new ThreadSync(d, "Samsun");
        ts1.start();
        ts2.start();
        ts3.start();
        ts4.start();
    }
}
```

Output:

Good morning
Rohit
Good morning
Rohit
Good morning
Rohit
Good morning
Rohit
Good morning
Rohit
Good morning
Rohit
Good morning
Rohit
Good morning
Rohit
Good morning
Rohit
Good morning
Rohit
Good morning
Samsun
Good morning
Samsun
Good morning
Samsun
Good morning
Samsun
Good morning
Samsun
Good morning
Samsun
Good morning
Samsun
Good morning
Samsun
Good morning
Samsun
Good morning
Samsun
Good morning
Surya
Good morning
Surya
Good morning
Surya
Good morning
Surya
Good morning
Surya
Good morning
Surya
Good morning
Surya
Good morning
Surya

[illegible]

Output is unpredictable, because threads are operating on different object. Even though `wish()` method is synchronized.

```
package ThreadSynchronization;
public class SynchronizedDemo {
    public static void main(String[] args) {
        Display d = new Display();
        Display d2 = new Display();
        ThreadSync ts1 = new ThreadSync(d, "Rohit");
        ThreadSync ts5 = new ThreadSync(d2, "Raghav");
        ts1.start();
        ts5.start();
    }
}
```

Output:

Good morning
Good morning
Rohit
Good morning
Raghav
Good morning
Raghav
Good morning
Rohit

If multiple threads are operating on same Java object then synchronization is required. If multiple threads are operating on multiple Java objects then

[illegible]

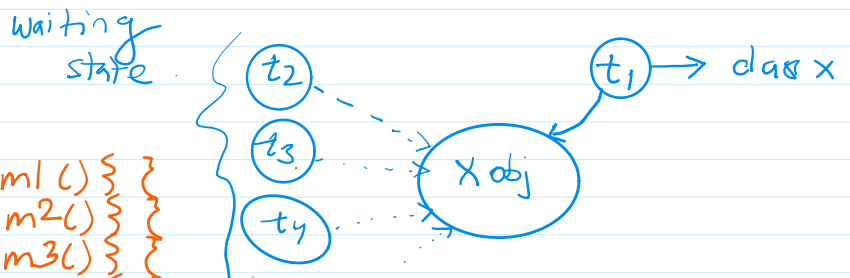
then synchronization is required.
If multiple threads are operating on multiple Java objects then synchronization is not required.

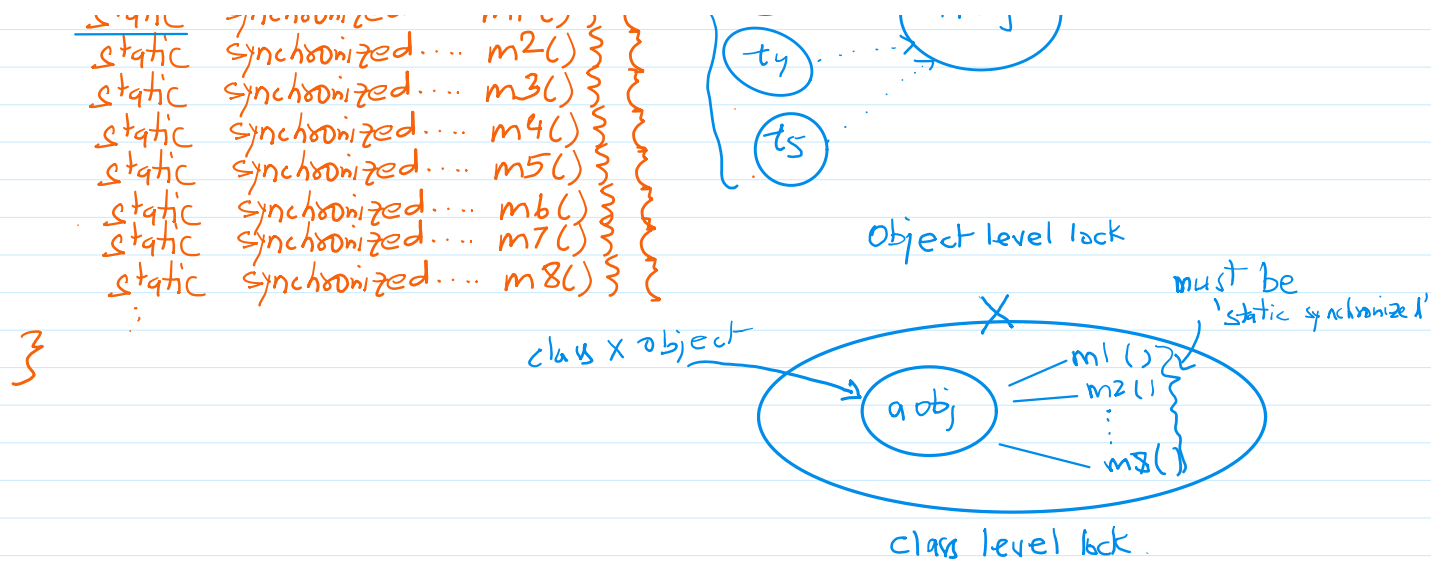
Class level lock

Every class in Java has a unique lock which is nothing but class level lock. If a thread wants to execute "static synchronized", then thread required class level lock. Once thread got class level lock, then it is allowed to execute any "static synchronized" method of that class. Once the method execution completes automatically thread releases the lock.

```
class X {
```

<u>static</u>	synchronized...	m1()	}
static	synchronized...	m2()	}
static	synchronized...	m3()	}





```
package ThreadSynchronization;
public class ClassX {
    public static synchronized void m1(){
        for (int i = 0; i < 5; i++) {
            System.out.println("m1(): " + Thread.currentThread().getName() + " Value : "
+ i);
            try {
                Thread.sleep(2000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
    public static synchronized void m2(){
        for (int i = 0; i < 5; i++) {
            System.out.println("m2(): " + Thread.currentThread().getName() + " Value : "
+ i);
            try {
                Thread.sleep(2000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
    public static synchronized void m3(){
        for (int i = 0; i < 5; i++) {
            System.out.println("m3(): " + Thread.currentThread().getName() + " Value : "
+ i);
            try {
                Thread.sleep(2000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
    public static synchronized void m4(){
```

```

        for (int i = 0; i < 5; i++) {
            System.out.println("m4(): " + Thread.currentThread().getName() + " Value : "
+ i);
            try {
                Thread.sleep(2000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
    public static synchronized void m5(){
        for (int i = 0; i < 5; i++) {
            System.out.println("m5(): " + Thread.currentThread().getName() + " Value : "
+ i);
            try {
                Thread.sleep(2000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

```

```

package ThreadSynchronization;
public class ThreadX extends Thread{
    @Override
    public void run() {
        ClassX.m1();
        ClassX.m2();
        ClassX.m3();
        ClassX.m4();
        ClassX.m5();
    }
}

```

```

package ThreadSynchronization;
public class MainX {
    public static void main(String[] args) {
        ThreadX t1 = new ThreadX();
        ThreadX t2 = new ThreadX();
        t1.start();
        t2.start();
    }
}

```

Output:

```

m1(): Thread-0 Value : 0
m1(): Thread-0 Value : 1
m1(): Thread-0 Value : 2
m1(): Thread-0 Value : 3
m1(): Thread-0 Value : 4
m2(): Thread-0 Value : 0
m2(): Thread-0 Value : 1
m2(): Thread-0 Value : 2
m2(): Thread-0 Value : 3

```

```

m2(): Thread-0 Value : 1
m2(): Thread-0 Value : 2
m2(): Thread-0 Value : 3
m2(): Thread-0 Value : 4
m3(): Thread-0 Value : 0
m3(): Thread-0 Value : 1
m3(): Thread-0 Value : 2
m3(): Thread-0 Value : 3
m3(): Thread-0 Value : 4
m4(): Thread-0 Value : 0
m4(): Thread-0 Value : 1
m4(): Thread-0 Value : 2
m4(): Thread-0 Value : 3
m4(): Thread-0 Value : 4
m5(): Thread-0 Value : 0
m5(): Thread-0 Value : 1
m5(): Thread-0 Value : 2
m5(): Thread-0 Value : 3
m5(): Thread-0 Value : 4
m1(): Thread-1 Value : 0
m1(): Thread-1 Value : 1
m1(): Thread-1 Value : 2
m1(): Thread-1 Value : 3
m1(): Thread-1 Value : 4
m2(): Thread-1 Value : 0
m2(): Thread-1 Value : 1
m2(): Thread-1 Value : 2
m2(): Thread-1 Value : 3
m2(): Thread-1 Value : 4
m3(): Thread-1 Value : 0
m3(): Thread-1 Value : 1
m3(): Thread-1 Value : 2
m3(): Thread-1 Value : 3
m3(): Thread-1 Value : 4
m4(): Thread-1 Value : 0
m4(): Thread-1 Value : 1
m4(): Thread-1 Value : 2
m4(): Thread-1 Value : 3
m4(): Thread-1 Value : 4
m5(): Thread-1 Value : 0
m5(): Thread-1 Value : 1
m5(): Thread-1 Value : 2
m5(): Thread-1 Value : 3
m5(): Thread-1 Value : 4

```

While a thread executing "static synchronized" method, the remaining threads are not allowed to execute any "static synchronized" method of that class simultaneously, but remaining threads are allowed to execute the following methods simultaneously:

1. Normal static method
2. Synchronized instance method.
3. Normal instance method.

```

package ThreadSynchronization;
public class MyNumber {
    public synchronized void displayNumber() throws InterruptedException{
        for (int i = 0; i < 5; i++) {
            System.out.print(i+" ");
            Thread.sleep(2000);
        }
    }
    public synchronized void displayCharacters() throws InterruptedException{

```

```

        for(char i = 'A'; i < 'K'; i++){
            System.out.print(i + " ");
            Thread.sleep(2000);
        }
    }
}

```

```

package ThreadSynchronization;
public class ThreadSync1 extends Thread {
    private MyNumber num;
    public ThreadSync1(MyNumber num) {
        this.num = num;
    }

    @Override
    public void run() {
        try {
            num.displayNumber();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

```

```

package ThreadSynchronization;
public class ThreadSync2 extends Thread {
    private MyNumber n;
    public ThreadSync2(MyNumber n){
        this.n = n;
    }
    @Override
    public void run() {
        try {
            n.displayCharacters();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

```

```

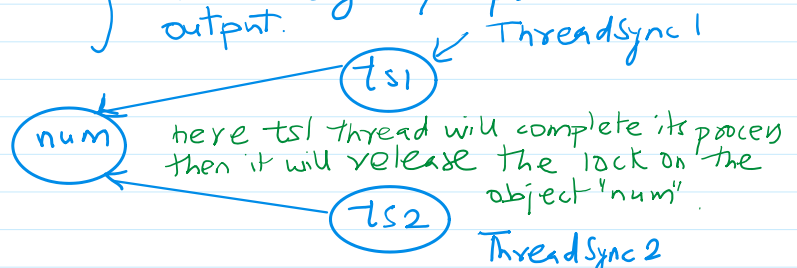
package ThreadSynchronization;
public class SynchronizedDemoMain {
    public static void main(String[] args) {
        MyNumber num = new MyNumber();
        ThreadSync1 ts1 = new ThreadSync1(num);
        ThreadSync2 ts2 = new ThreadSync2(num);
        ts1.start();
        ts2.start();
    }
}

```

Output:

```
0 1 2 3 4 A B C D E F G H I J
```

Different threads occupy same class object and calling different synchronized method from same class will give you predictable output.



752

object 'num'.

ThreadSync 2

1. To get lock of current object

```
synchronized(this){
    //code
}
```

← If a thread gets lock of current object then only it is allowed to execute on current object.

2. To get lock of a particular object say 'x'

```
synchronized(x){
    //code
}
```

← If a thread gets lock of particular object 'x' then only it is allowed to execute this area.

3. To get class level lock

```
synchronized(MyNumber.class){
    //code
}
```

← If a thread got class level lock of "MyNumber" class then only it is allowed to execute this area.

```
package ThreadSynchronization;
public class DisplayWish {
    public void wish(String name){
        ;;;;;;;;;; //million line code
        ;;;;;;;;;;
        //synchronized(this){ //using the current object
        // synchronized(new DisplayWish()){ //object level lock
        synchronized(DisplayWish.class){ //class level lock

            for (int i = 0; i < 10; i++) {
                System.out.println("Good morning: " + name);
            }
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        ;;;;;;;;;; //million line of code
        ;;;;;;;;;;
```

as object level lock is occupied,
so synchronized block will work for same object.

```

    {
        ;;;;;;;;;;;;;//million line of code
        ;;;;;;;;;;;;;
    }
}

```

as object level lock is occupied,
so synchronized block will work for
same object.

Solution is use current object concept
i.e. `synchronized(this)` {

```

package ThreadSynchronization;
public class Thread100 extends Thread{
    private DisplayWish dw;
    private String name, x;
    public Thread100(DisplayWish d, String name){
        this.dw = d;
        this.name = name;
    }
    public Thread100(String a, String b){
        name = a;
        x = b;
    }
    @Override
    public void run() {
        dw.wish(name);
    }
}

```

}
block.

```

package ThreadSynchronization;
public class SynchronizedDemo100 {
    public static void main(String[] args) {
        DisplayWish dw = new DisplayWish();
        Thread100 t1 = new Thread100(dw, "Arvind Kejriwal");
        Thread100 t2 = new Thread100(dw, "Mamta Banerjee");
        Thread100 t3 = new Thread100(dw, "Yogi Aditya Nath");
        t1.start();
        t2.start();
        t3.start();
    }
}

```

1. What is synchronize keyword? Where we can apply?

Ans: Synchronization keyword is used in Java to prevent data inconsistency and to provide controlled access to critical sections by various threads.

We can apply this keyword in methods and blocks.

2. Explain advantage of synchronized keywords?

Ans: Resolves data inconsistency problem caused by multiple threads trying to access their critical section simultaneously.

3. Explain disadvantage of synchronized keyword.

Ans: It increases waiting time of other threads and hence creates performance problem. Hence it is not recommended to use if there is no specific requirement.

4. What is Race condition?(Data inconsistency)

Ans: When multiple threads are trying to operate simultaneously on a single Java object, the outcome depends on the order in which they execute. Hence the execution order becomes unpredictable, thus producing inconsistent results. This is called race condition.

5. What is object level lock and when it is required?

Ans: It is associated with a specific instance of a class. Ensures only one thread at a time can execute synchronized code on that object. It doesn't effect other objects.

6. What is class level lock and when it is required?

Ans: Every class in Java has this unique lock. It is associated with the class object itself. If a thread wants to execute "static synchronized" methods, then it requires class level lock. Can also be used by `synchronized(ClassName.class)` blocks.

We use it when we want to access or modify class variables. And when we want only a single thread of entire JVM to access static synchronized method of the class at a time.

7. What is the difference between class level lock and object level lock?

Ans:

Class level

- Provides global synchronization.
- Applies to all instances of that class.
- Used with "static synchronized" method.
- Can also be declared using `synchronized(ClassName.class) {}`

Object level

- Applies to only a particular instance of A class.
- Used with synchronized instance methods.
- Can be declared using `synchronized (class object_name) {}` Or `synchronized(this) {}`

8. While a thread executing synchronized method on the given object is remaining threads are allowed to execute or any other synchronized thread simultaneously on the same object.

Ans: No

9. What is synchronized block?

Ans: It's a block that can be accessed by only one thread at a time in whole JVM.

10. How to declare synchronized block to get lock of current object?

Ans: `synchronized(this){}`

11. How to declare synchronized block to get lock of class level lock?

Ans: `synchronized(ClassName.class){...}`

12. What are the advantages of synchronized block over synchronized method?

Ans:

- Locks only a particular section instead of entire method.
- Locks on any object user specifies.
- Allows other threads to access non critical reason.
- Provides more flexibility to the user.

13. Is a thread can acquire multiple locks simultaneously? Explain with a sample code.

Ans: A thread can acquire multiple locks simultaneously, but risk deadlock if order mismatch across threads. Use consistent lock ordering to avoid.

```
package ThreadSynchronization;
public class DeadLockExample {
    private final Object lock1 = new Object();
    private final Object lock2 = new Object();
    public static void main(String[] args) {
        DeadLockExample dle = new DeadLockExample();
        Thread t1 = new Thread(dle::method1, "Thread-1");
        Thread t2 = new Thread(dle::method2, "Thread-2");
        t1.start();
        t2.start();
    }
    void method1(){
        synchronized(lock1){
            System.out.println("Thread-1: holding lock-1");
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            System.out.println("Thread-1: Waiting for lock-2");
            synchronized(lock2){
                System.out.println("Thread-1: holding lock-1 and lock-2");
            }
        }
    }
    void method2(){
        synchronized(lock2){
            System.out.println("Thread-2: holding lock2");
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            System.out.println("Thread-2 waiting for lock1");
            synchronized(lock1){
                System.out.println("Thread-2: holding lock1 and lock2");
            }
        }
    }
}
```

Order mismatch
deadlock has
happened.
lock1 → lock2 is
occupied by method1().
lock2 → lock1 is
occupied by method2()

Order maintained:

```
package ThreadSynchronization;
public class DeadLockExample {
    private final Object lock1 = new Object();
    private final Object lock2 = new Object();
    public static void main(String[] args) {
        DeadLockExample dle = new DeadLockExample();
        Thread t1 = new Thread(dle::method1, "Thread-1");
        Thread t2 = new Thread(dle::method2, "Thread-2");
        t1.start();
        t2.start();
    }
    void method1(){
```

```

        t2.start();
    }
    void method1(){
        synchronized(lock2){
            System.out.println("Thread-1: holding lock-1");
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            System.out.println("Thread-1: Waiting for lock-2");
            synchronized(lock1){
                System.out.println("Thread-1: holding lock-1 and lock-2");
            }
        }
    }
    void method2(){
        synchronized(lock2){
            System.out.println("Thread-2: holding lock2");
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            System.out.println("Thread-2 waiting for lock1");
            synchronized(lock1){
                System.out.println("Thread-2: holding lock1 and lock2");
            }
        }
    }
}

```

← Order of
 lock is maintained
 There is no deadlock.
 lock 2 → lock1 in
 method1()
 lock 2 → lock1 in
 method2()

What is synchronized statement? (Interview created terminology)

The statements present in synchronized method and synchronized block are called synchronized statements.